Problem Statement:

Implement FCFS CPU Scheduling for the following scenario where No of processes = 5 and Process informations are-

P id	AT	BT
P1	3	3
P2	2	5
P3	5	4
P4	1	3
P5	6	2

Calculate CT, TAT, WT, AVG TAT and AVG WT. Also represent the process execution sequence.

SOURCE CODE :

```
p_id = ["p1", "p2", "p3", "p4", "p5"]

at = [3, 2, 5, 1, 6]

bt = [3, 5, 4, 3, 2]


fcfs_id = p_id.copy()

fcfs_at = at.copy()

fcfs_bt = bt.copy()


for i in range(len(fcfs_at)-1):
```

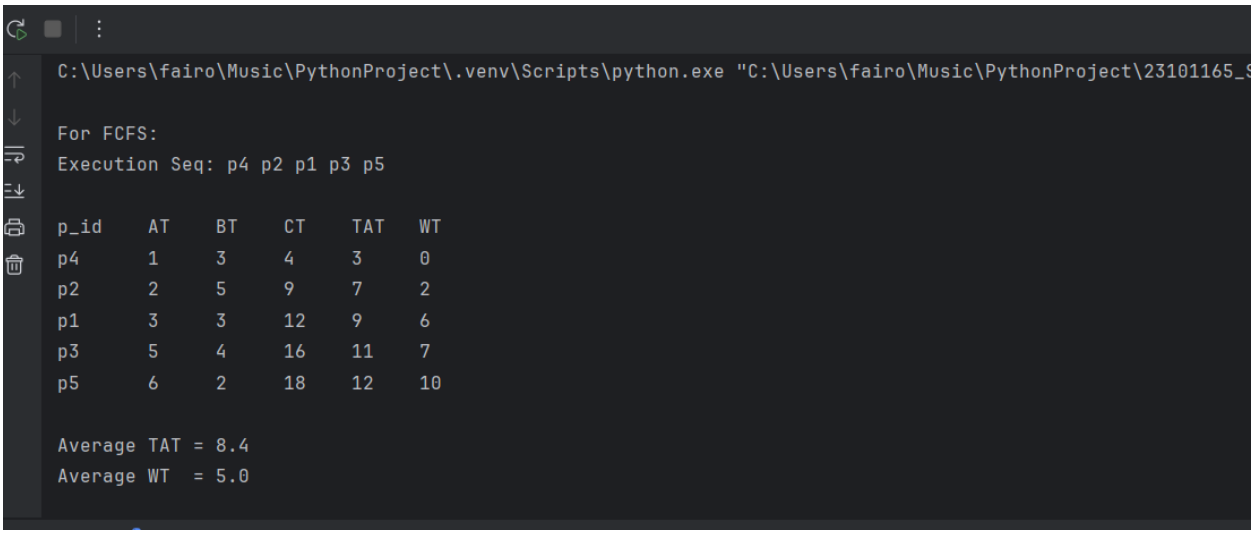
```
    for j in range(len(fcfs_at)-1-i):
        if fcfs_at[j] > fcfs_at[j+1]:
            fcfs_at[j], fcfs_at[j+1] = fcfs_at[j+1], fcfs_at[j]
            fcfs_bt[j], fcfs_bt[j+1] = fcfs_bt[j+1], fcfs_bt[j]
            fcfs_id[j], fcfs_id[j+1] = fcfs_id[j+1], fcfs_id[j]

fcfs_ct = []
fcfs_tat = []
fcfs_wt = []

current_time = 0
sum_tat = 0
sum_wt = 0

for i in range(len(fcfs_id)):

    if current_time < fcfs_at[i]:
        current_time = fcfs_at[i]

    current_time = current_time + fcfs_bt[i]

    fcfs_ct.append(current_time)
    fcfs_tat.append(fcfs_ct[i] - fcfs_at[i])
    fcfs_wt.append(fcfs_tat[i] - fcfs_bt[i])

    sum_tat = sum_tat + fcfs_tat[i]
    sum_wt = sum_wt + fcfs_wt[i]
```

```
fcfs_avg_tat = sum_tat / len(fcfs_id)
fcfs_avg_wt = sum_wt / len(fcfs_id)

sjf_id = p_id.copy()
sjf_at = at.copy()
sjf_bt = bt.copy()

sjf_ct = [0, 0, 0, 0, 0]
sjf_tat = [0, 0, 0, 0, 0]
sjf_wt = [0, 0, 0, 0, 0]
done = [0, 0, 0, 0, 0]
time = 0
completed = 0
sjf_seq = []

while completed < len(sjf_id):

    smallest = -1

    for i in range(len(sjf_id)):
        if sjf_at[i] <= time and done[i] == 0:

            if smallest == -1 or sjf_bt[i] < sjf_bt[smallest]:
                smallest = i

    if smallest != -1:
```

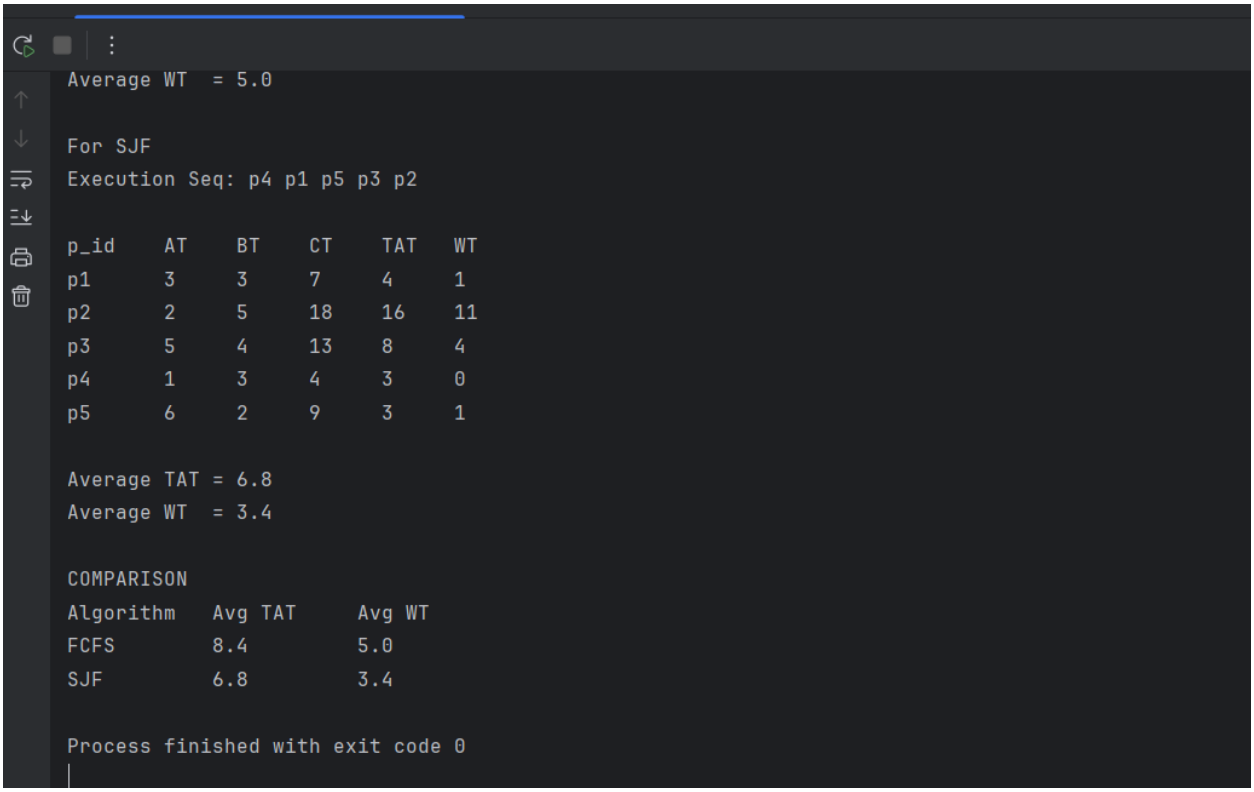
```
        sjf_seq.append(sjf_id[smallest])
        time = time + sjf_bt[smallest]
        sjf_ct[smallest] = time
        sjf_tat[smallest] = sjf_ct[smallest] - sjf_at[smallest]
        sjf_wt[smallest] = sjf_tat[smallest] - sjf_bt[smallest]
        done[smallest] = 1
        completed = completed + 1

    else:
        time = time + 1



sjf_avg_tat = sum(sjf_tat) / len(sjf_id)
sjf_avg_wt = sum(sjf_wt) / len(sjf_id)

print("\nFor FCFS:")
print("Execution Seq:", end=" ")
for i in range(len(fcfs_id)):
    print(fcfs_id[i], end=" ")

print("\n")
print(f"{'p_id':<8}{'AT':<6}{'BT':<6}{'CT':<6}{'TAT':<6}{'WT':<6}")

for i in range(len(fcfs_id)):
    print(
        f"{fcfs_id[i]:<8}{fcfs_at[i]:<6}{fcfs_bt[i]:<6}"
```

```
        f"{fcfs_ct[i]:<6}{fcfs_tat[i]:<6}{fcfs_wt[i]:<6}"
    )

print("\nAverage TAT =", fcfs_avg_tat)
print("Average WT  =", fcfs_avg_wt)

print("\nFor SJF")
print("Execution Seq:", end=" ")
for process in sjf_seq:
    print(process, end=" ")

print("\n")
print(f"{'p_id':<8}{'AT':<6}{'BT':<6}{'CT':<6}{'TAT':<6}{'WT':<6}")

for i in range(len(sjf_id)):
    print(
        f"{sjf_id[i]:<8}{sjf_at[i]:<6}{sjf_bt[i]:<6}"
        f"{sjf_ct[i]:<6}{sjf_tat[i]:<6}{sjf_wt[i]:<6}"
    )

print("\nAverage TAT =", sjf_avg_tat)
print("Average WT  =", sjf_avg_wt)
print("\nCOMPARISON")
print(f"{'Algorithm':<12}{'Avg TAT':<12}{'Avg WT':<12}")
print(f"{'FCFS':<12}{fcfs_avg_tat:<12.1f}{fcfs_avg_wt:<12.1f}")
print(f"{'SJF':<12}{sjf_avg_tat:<12.1f}{sjf_avg_wt:<12.1f}")
```

OUTPUT :



```
C:\Users\fairo\Music\PythonProject\.venv\Scripts\python.exe "C:\Users\fairo\Music\PythonProject\23101165_S

For FCFS:
Execution Seq: p4 p2 p1 p3 p5

p_id    AT    BT    CT    TAT   WT
p4      1     3     4     3     0
p2      2     5     9     7     2
p1      3     3     12    9     6
p3      5     4     16    11    7
p5      6     2     18    12    10

Average TAT = 8.4
Average WT  = 5.0
```



```
Average WT  = 5.0

For SJF
Execution Seq: p4 p1 p5 p3 p2

p_id    AT    BT    CT    TAT   WT
p1      3     3     7     4     1
p2      2     5     18    16    11
p3      5     4     13    8     4
p4      1     3     4     3     0
p5      6     2     9     3     1

Average TAT = 6.8
Average WT  = 3.4

COMPARISON
Algorithm   Avg TAT     Avg WT
FCFS        8.4         5.0
SJF         6.8         3.4

Process finished with exit code 0
```

DISCUSSION:

In this experiment, FCFS (First Come First Serve) and SJF (Shortest Job First) CPU scheduling algorithms were implemented in Python and their performances were

compared using Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT).

While implementing the program, I faced several challenges. First, I had to maintain the original process information while sorting the processes according to their Arrival Time for FCFS. To solve this, I created copies of the Process ID, Arrival Time, and Burst Time lists using the copy() method. I then used Bubble Sort to arrange the processes according to Arrival Time. While swapping the Arrival Time, I also swapped the corresponding Burst Time and Process ID so that the process information remained consistent.

Another challenge was calculating the timing values correctly. Initially, it was difficult to keep track of the CPU's current execution time and correctly calculate CT, TAT, and WT for every process. I solved this by introducing the current_time variable for FCFS. Before executing each process, I checked whether the CPU was idle. If the current time was less than the process's Arrival Time, I updated the current time to the Arrival Time of that process. Then, the Burst Time was added to calculate the Completion Time. Using the calculated Completion Time, I obtained TAT and WT.

Implementing SJF was more challenging because, unlike FCFS, the processes cannot simply be arranged once according to Arrival Time. At every stage, I had to check which processes had already arrived and then select the process with the shortest Burst Time. To handle this, I used the done list to keep track of completed processes. I also used the smallest variable to identify the process with the shortest Burst Time among the available processes. The completed variable was used to determine when all processes had finished execution.

I also had to consider the situation where no process was available at the current time. In that case, the value of smallest remained -1. To handle this situation, I increased the time variable by one until a process became available. This helped me correctly handle CPU idle time and prevented the program from getting stuck.

From the output, FCFS produced an execution sequence of p4 → p2 → p1 → p3 → p5, with an average Turnaround Time of **8.4** and an average Waiting Time of **5.0**. On the other hand, SJF produced the execution sequence p4 → p1 → p5 → p3 → p2, with an average Turnaround Time of **6.8** and an average Waiting Time of **3.4**. Therefore, for the given set of processes, SJF performed better than FCFS in terms of both average TAT and average WT.

Why is FCFS required in CPU Scheduling?

FCFS is required because it is one of the simplest and fairest CPU scheduling approaches. It executes processes in the order in which they arrive in the ready queue. It is easy to implement and understand, and it does not require complex calculations or

continuous comparison of process characteristics. FCFS can be useful when simplicity, predictable ordering, and low scheduling overhead are more important than minimizing waiting time. However, it can suffer from the **convoy effect**, where a long process makes shorter processes wait for a long time.

When does SJF perform better than FCFS?

SJF generally performs better than FCFS when processes have different Burst Times, particularly when shorter processes are available while a longer process is waiting. By selecting the process with the shortest Burst Time first, SJF can reduce the waiting time of shorter processes and often results in a lower average Waiting Time and Turnaround Time. In this experiment, SJF performed better because its average Waiting Time was **3.4**, compared with **5.0** for FCFS, and its average Turnaround Time was **6.8**, compared with **8.4** for FCFS. Therefore, SJF was more efficient for the given process set.

CONCLUSION:

This experiment provided practical experience in implementing and comparing the FCFS and SJF CPU scheduling algorithms using Python. During the implementation, I faced difficulties with sorting the processes, maintaining the relationship between Process ID, Arrival Time, and Burst Time, calculating the scheduling times, and selecting the shortest available process in SJF. I solved these problems by using Bubble Sort for FCFS, separate tracking variables such as current_time and time, and the done, smallest, and completed variables for SJF.

The comparison of the results shows that SJF performed better than FCFS for the given set of processes. FCFS resulted in an average TAT of 8.4 and an average WT of 5.0, whereas SJF resulted in an average TAT of 6.8 and an average WT of 3.4. This indicates that SJF was able to reduce both the average waiting time and turnaround time by prioritizing shorter available processes.

However, FCFS remains useful because of its simplicity, fairness, and ease of implementation. SJF can provide better performance when reducing average waiting and turnaround time is the main objective, but it requires more decision-making during scheduling and may cause longer processes to wait for a significant amount of time. Overall, this experiment helped me understand the practical differences between FCFS and SJF and improved my programming, debugging, and problem-solving skills.